

# Python Programming Complete Study Guide

Every topic explained with example code and real output

13 Units | 56 Topics | Core Python, OOP, Advanced Topics, Pandas

All code examples were executed and the outputs shown are the actual results.

# Contents

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Unit 1: Introduction to Python</b>                                        | <b>6</b>  |
| Keywords and Identifiers . . . . .                                           | 6         |
| Statements, Indentation and Comments . . . . .                               | 6         |
| Python Variables . . . . .                                                   | 7         |
| Python Type Conversion . . . . .                                             | 7         |
| Python Input / Output and import . . . . .                                   | 8         |
| Python Operators . . . . .                                                   | 8         |
| Python Namespace and Scope . . . . .                                         | 9         |
| <b>Unit 2: Python Data Types - Numbers, List, Tuple</b>                      | <b>10</b> |
| Python Numbers . . . . .                                                     | 10        |
| Python List . . . . .                                                        | 10        |
| Python Tuple . . . . .                                                       | 11        |
| <b>Unit 3: Python Data Types - String, Set, Frozenset, Dictionary, Range</b> | <b>12</b> |
| Python String . . . . .                                                      | 12        |
| Python Set . . . . .                                                         | 12        |
| Python Frozenset . . . . .                                                   | 13        |
| Python Dictionary . . . . .                                                  | 14        |
| Python range . . . . .                                                       | 14        |
| <b>Unit 4: Flow Control and Conditional Statements</b>                       | <b>16</b> |
| Python if...else...elif . . . . .                                            | 16        |
| Python for Loop . . . . .                                                    | 16        |
| Python while Loop . . . . .                                                  | 17        |
| Python break and continue . . . . .                                          | 18        |
| Python pass . . . . .                                                        | 18        |
| <b>Unit 5: Python Functions</b>                                              | <b>20</b> |
| Python Function . . . . .                                                    | 20        |
| Function Arguments . . . . .                                                 | 20        |
| Python Recursion . . . . .                                                   | 21        |
| Anonymous (Lambda) Function . . . . .                                        | 21        |
| Global, Local and Nonlocal Variables . . . . .                               | 22        |
| Python global Keyword . . . . .                                              | 22        |

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Unit 6: Python Object and Class (OOP)</b>                                 | <b>24</b> |
| Python OOPs Concepts . . . . .                                               | 24        |
| Classes and Objects . . . . .                                                | 24        |
| Inheritance . . . . .                                                        | 25        |
| Abstraction and Encapsulation . . . . .                                      | 25        |
| Overloading and Overriding . . . . .                                         | 26        |
| <b>Unit 7: Python Methods, Variables and Constructors</b>                    | <b>28</b> |
| Instance Method, Class Method and Static Method . . . . .                    | 28        |
| Instance Variables and Class Variables . . . . .                             | 28        |
| Constructors . . . . .                                                       | 29        |
| <b>Unit 8: Python Files and Exception Handling</b>                           | <b>30</b> |
| Python File Operations and File Handling . . . . .                           | 30        |
| Exception Handling . . . . .                                                 | 31        |
| <b>Unit 9: Python Advanced Topics</b>                                        | <b>32</b> |
| Python Iterator . . . . .                                                    | 32        |
| Python Generator . . . . .                                                   | 32        |
| Python Decorators . . . . .                                                  | 33        |
| lambda, map, filter and reduce . . . . .                                     | 34        |
| <b>Unit 10: Python Modules, Packages, Libraries, Regex and Datetime</b>      | <b>35</b> |
| Python Modules . . . . .                                                     | 35        |
| Python Packages . . . . .                                                    | 35        |
| Python Libraries . . . . .                                                   | 36        |
| Python Regex (re module) . . . . .                                           | 36        |
| Python Date and Time . . . . .                                               | 37        |
| <b>Unit 11: Pandas DataFrames - Introduction, Selecting and Viewing Data</b> | <b>38</b> |
| Introduction to Pandas and DataFrames . . . . .                              | 38        |
| Viewing Data . . . . .                                                       | 38        |
| Selecting Data: columns, loc, iloc, filtering . . . . .                      | 40        |
| <b>Unit 12: Operations, Grouping, Merging and Manipulating Data</b>          | <b>41</b> |
| Data Operations and Cleaning . . . . .                                       | 41        |
| Manipulating Data: columns, sorting, pivot . . . . .                         | 41        |
| Grouping Data (groupby) . . . . .                                            | 42        |
| Merging, Joining and Concatenating . . . . .                                 | 43        |

|                                                                               |               |
|-------------------------------------------------------------------------------|---------------|
| <b>Unit 13: Reading Files using Python and Pandas (CSV, JSON, Excel, XML)</b> | <b>45</b>     |
| Reading CSV Files . . . . .                                                   | 45            |
| Reading JSON Files . . . . .                                                  | 45            |
| Reading Excel Files . . . . .                                                 | 46            |
| Reading XML Files . . . . .                                                   | 47            |
| <br><b>Quick Revision: Differences Asked in Exams</b>                         | <br><b>48</b> |
| List vs Tuple vs Set vs Dictionary . . . . .                                  | 48            |
| Other Common Differences . . . . .                                            | 48            |
| <br><b>Practice Programs Checklist</b>                                        | <br><b>49</b> |

## How to Use This Guide

- Each topic has three parts: a short **explanation** (what it is and the key rules), an **example program** (blue label) and the **output** it produces (green label).
- **Do not just read.** Type every example yourself, change the values, and predict the output before running it. Typing and debugging is what builds exam and assignment skill.
- For written answers, follow this structure: **definition, then a short code example, then the output.** The final section lists the classic 'difference between' questions in table form.
- Examples that read files create their own sample files, so every program runs as-is. The Pandas examples need *pip install pandas openpyxl lxml*.
- Note: outputs were produced with Python 3.12 and pandas 3.0. On older pandas (2.x) text columns show the dtype *object* instead of *str*, and the DataFrame class prints as *pandas.core.frame.DataFrame*. Nothing else changes.

# Unit 1: Introduction to Python

## Keywords and Identifiers

**Keywords** are reserved words with a special meaning in Python (e.g. *if*, *else*, *for*, *while*, *def*, *class*, *True*, *None*). They cannot be used as variable names. **Identifiers** are names given to variables, functions, classes, etc. Rules: they may contain letters, digits and underscore; cannot start with a digit; cannot be a keyword; and are case-sensitive (*age* and *Age* are different).

### EXAMPLE CODE

```
import keyword
print(len(keyword.kwlist), "keywords")
print(keyword.kwlist[:8])

student_name = "Asha"      # valid
_count = 10                # valid
total2 = 50                # valid
# 2total = 5               # INVALID: starts with a digit
# for = 3                  # INVALID: keyword
print(keyword.iskeyword("for"), keyword.iskeyword("Name"))
```

### OUTPUT

```
35 keywords
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await']
True False
```

## Statements, Indentation and Comments

A **statement** is an instruction Python executes (assignment, *print*, *if*, etc.). A long statement can continue on the next line using a backslash `\` or brackets. Python uses **indentation** (usually 4 spaces) instead of braces to define blocks. **Comments** start with `#` and are ignored by Python; a triple-quoted string placed right after a `def/class` is a **docstring** used for documentation.

### EXAMPLE CODE

```
# This is a single-line comment
a = 1 + 2 + 3 + \
    4 + 5                # line continuation with backslash
b = (10 +
    20 +
    30)                  # continuation inside brackets
x = 5; y = 6             # two statements on one line
print(a, b, x, y)

def greet():
    """This is a docstring: it describes the function."""
    print("Hello")        # indented block
greet()
print(greet.__doc__)
```

### OUTPUT

```
15 60 5 6
Hello
This is a docstring: it describes the function.
```

## Python Variables

A variable is a name that refers to a value stored in memory. Python is **dynamically typed**: you do not declare a type, and the same name can later refer to a different type. Use `type()` to check the type and `id()` to see the memory identity. Multiple assignment and swapping are very convenient. Constants are written in UPPER\_CASE by convention.

### EXAMPLE CODE

```
x = 10
print(type(x))
x = "now a string"
print(type(x))

a, b, c = 1, 2.5, "hi"      # multiple assignment
p = q = 0                  # same value to many names
a, b = b, a                 # swap without temp variable
print(a, b, c, p, q)

PI = 3.14159               # constant by convention
print(id(p) == id(q))      # both point to same object
```

### OUTPUT

```
<class 'int'>
<class 'str'>
2.5 1 hi 0 0
True
```

## Python Type Conversion

Converting one data type into another. **Implicit conversion** is done automatically by Python (e.g. int + float gives float). **Explicit conversion** (type casting) is done by you using `int()`, `float()`, `str()`, `bool()`, `list()`, `tuple()`, `set()`. Invalid conversions raise `ValueError`.

### EXAMPLE CODE

```
# Implicit
print(5 + 2.0, type(5 + 2.0))

# Explicit
print(int("42") + 1)
print(float(7), str(3.5) + "!")
print(int(9.99))          # truncates, does not round
print(list("abc"), tuple([1, 2]), set([1, 1, 2]))
print(bool(0), bool("text"), bool([]))

try:
    int("abc")
except ValueError as e:
    print("Error:", e)
```

### OUTPUT

```
7.0 <class 'float'>
43
7.0 3.5!
9
['a', 'b', 'c'] (1, 2) {1, 2}
False True False
Error: invalid literal for int() with base 10: 'abc'
```

## Python Input / Output and import

`print()` displays output; its *sep* and *end* arguments control separators. `input()` reads text from the user and **always returns a string**, so convert it with `int()`/`float()` when needed. Formatting can be done with f-strings, `format()` or `%`. The `import` statement brings in modules; use *from module import name* or *import module as alias*.

### EXAMPLE CODE

```
name = input("Enter name: ")
age = int(input("Enter age: "))
print(f"Hello {name}, next year you will be {age + 1}")
print("a", "b", "c", sep="-", end="\n")
print("{} scored {:.1f}".format("Ravi", 87.456))

import math
from random import randint
import datetime as dt
print(math.sqrt(16), math.pi)
```

### OUTPUT

```
Enter name: Enter age: Hello Asha, next year you will be 21
a-b-c!
Ravi scored 87.5
4.0 3.141592653589793
```

*Sample run: the user typed Asha and 20 at the two prompts.*

## Python Operators

Operators perform operations on values. Types: **Arithmetic** (+ - \* / // % \*\*), **Comparison** (== != < > <= >=), **Logical** (and, or, not), **Bitwise** (& | ^ ~ << >>), **Assignment** (= += -= \*= etc.), **Identity** (is, is not) and **Membership** (in, not in). Precedence (high to low): \*\*, unary, \* / // %, + -, comparisons, not, and, or.

### EXAMPLE CODE

```
a, b = 17, 5
print(a + b, a - b, a * b, a / b, a // b, a % b, a ** 2)
print(a > b, a == b, a != b)
print(a > 10 and b < 3, a > 10 or b < 3, not a > 10)
print(a & b, a | b, a ^ b, a << 1, a >> 1) # bitwise
a += 3; print(a) # assignment operator

x = [1, 2]; y = [1, 2]
print(x == y, x is y) # equal values, different objects
print(3 in x, 2 in x) # membership
print(2 + 3 * 4 ** 2) # precedence -> 50
```

### OUTPUT

```
22 12 85 3.4 3 2 289
True False True
False True False
1 21 20 34 8
20
True False
False True
50
```

## Python Namespace and Scope

A **namespace** is a mapping from names to objects. Python has: **Built-in** (print, len), **Global** (module level), **Enclosing** (outer function) and **Local** (inside a function) namespaces. Name lookup follows the **LEGB rule**: Local, Enclosing, Global, Built-in. *globals()* and *locals()* show the current namespaces.

### EXAMPLE CODE

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print("inner:", x)
    inner()
    print("outer:", x)

outer()
print("module:", x)

def show():
    y = 5
    print(list(locals().keys()))
show()
print("x" in globals())
```

### OUTPUT

```
inner: local
outer: enclosing
module: global
['y']
True
```

# Unit 2: Python Data Types - Numbers, List, Tuple

## Python Numbers

Python has three numeric types: **int** (unlimited size), **float** (decimal numbers) and **complex** ( $a + bj$ ). Useful functions: *abs*, *round*, *pow*, *divmod*, *max*, *min* and the *math* module. Numbers can be written in binary (0b), octal (0o) and hex (0x).

### EXAMPLE CODE

```
a = 10          # int
b = 3.14        # float
c = 2 + 3j      # complex
print(type(a), type(b), type(c))
print(c.real, c.imag, abs(c))
print(0b1010, 0o17, 0xFF)      # binary, octal, hex
print(2 ** 100)                # big integers are fine
print(round(3.14159, 2), abs(-7), pow(2, 5), divmod(17, 5))

import math
print(math.floor(4.7), math.ceil(4.2), math.factorial(5), math.gcd(12, 18))
print(0.1 + 0.2 == 0.3, math.isclose(0.1 + 0.2, 0.3))
```

### OUTPUT

```
<class 'int'> <class 'float'> <class 'complex'>
2.0 3.0 3.605551275463989
10 15 255
1267650600228229401496703205376
3.14 7 32 (3, 2)
4 5 120 6
False True
```

## Python List

A list is an **ordered**, **mutable** collection that allows duplicates and mixed types, written with square brackets. It supports indexing, negative indexing, slicing [*start:stop:step*] and many methods: *append*, *extend*, *insert*, *remove*, *pop*, *index*, *count*, *sort*, *reverse*, *copy*, *clear*. List comprehensions build lists in one line.

### EXAMPLE CODE

```
nums = [5, 2, 9, 1]
nums.append(7)          # add at end
nums.insert(1, 100)     # add at index
nums.extend([3, 3])     # add many
print(nums)
nums.remove(100)        # remove by value
print(nums.pop(), nums.pop(0)) # remove by index
nums.sort(); print(nums)
nums.reverse(); print(nums)

print(nums[0], nums[-1], nums[1:3], nums[::-1])
print(len(nums), nums.count(3), nums.index(9), max(nums), sum(nums))

squares = [n ** 2 for n in range(1, 6)]
even = [n for n in range(10) if n % 2 == 0]
print(squares, even)

a = [1, 2]; b = a.copy(); b.append(3)    # copy keeps a unchanged
print(a, b)
matrix = [[1, 2], [3, 4]]
print(matrix[1][0])
```

### OUTPUT

```
[5, 100, 2, 9, 1, 7, 3, 3]
3 5
[1, 2, 3, 7, 9]
[9, 7, 3, 2, 1]
```

```

9 1 [7, 3] [1, 2, 3, 7, 9]
5 1 0 9 22
[1, 4, 9, 16, 25] [0, 2, 4, 6, 8]
[1, 2] [1, 2, 3]
3

```

## Python Tuple

A tuple is an **ordered, immutable** collection written with parentheses. Once created, items cannot be changed, so tuples are faster than lists and can be used as dictionary keys. A single-element tuple needs a trailing comma. Tuples support indexing, slicing, *count()*, *index()* and **packing/unpacking**.

### EXAMPLE CODE

```

t = (10, 20, 30, 20)
print(t[0], t[-1], t[1:3], t.count(20), t.index(30))

single = (5,)          # comma is required
print(type(single), type((5)))

a, b, c, d = t          # unpacking
print(a, d)
first, *rest = t        # extended unpacking
print(first, rest)

try:
    t[0] = 99
except TypeError as e:
    print("Error:", e)

lst = list(t); lst.append(40); t = tuple(lst)  # "modify" by converting
print(t)

```

### OUTPUT

```

10 20 (20, 30) 2 2
<class 'tuple'> <class 'int'>
10 20
10 [20, 30, 20]
Error: 'tuple' object does not support item assignment
(10, 20, 30, 20, 40)

```

# Unit 3: Python Data Types - String, Set, Frozenset, Dictionary, Range

## Python String

A string is an **ordered, immutable** sequence of characters, written in single, double or triple quotes. Supports indexing and slicing. Common methods: *upper*, *lower*, *title*, *strip*, *split*, *join*, *replace*, *find*, *count*, *startswith*, *endswith*, *isdigit*, *isalpha*. Use **f-strings** for formatting.

### EXAMPLE CODE

```
s = " Hello, Python World "
print(s.strip())
print(s.upper(), s.lower().strip())
t = s.strip()
print(t[0], t[-1], t[0:5], t[::-1])
print(t.replace("Python", "Java"))
print(t.split(","))
print("-".join(["a", "b", "c"]))
print(t.find("Python"), t.count("o"), t.startswith("Hello"))
print("123".isdigit(), "abc".isalpha(), "Ab1".isalnum())

name, mark = "Ravi", 91.5
print(f"{name} scored {mark:.2f}")
print(len(t), "Py" in t)
print("""Multi
line""")
```

### OUTPUT

```
Hello, Python World
HELLO, PYTHON WORLD hello, python world
H d Hello dlrow nohtyP ,olleH
Hello, Java World
['Hello', ' Python World']
a-b-c
7 3 True
True True True
Ravi scored 91.50
19 True
Multi
line
```

## Python Set

A set is an **unordered collection of unique elements**, written with curly braces. It is mutable, but its elements must be immutable. Sets do not support indexing. Methods: *add*, *update*, *remove*, *discard*, *pop*, *clear*. Mathematical operations: **union (|)**, **intersection (&)**, **difference (-)**, **symmetric difference (^)**. Create an empty set with *set()*, not *{}*.

#### EXAMPLE CODE

```
s = {1, 2, 3, 3, 2}
print(s)                # duplicates removed
s.add(4); s.update([5, 6])
s.remove(6)              # error if missing
s.discard(99)            # no error if missing
print(s)

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B, A.union(B))
print(A & B, A.intersection(B))
print(A - B, A.difference(B))
print(A ^ B)
print({1, 2} <= A, A.isdisjoint({9}))

print(list(set([1, 1, 2, 3, 3]))) # remove duplicates from list
print(type({}), type(set()))
```

#### OUTPUT

```
{1, 2, 3}
{1, 2, 3, 4, 5}
{1, 2, 3, 4, 5, 6} {1, 2, 3, 4, 5, 6}
{3, 4} {3, 4}
{1, 2} {1, 2}
{1, 2, 5, 6}
True True
[1, 2, 3]
<class 'dict'> <class 'set'>
```

## Python Frozenset

A **frozenset** is an **immutable** version of a set. It supports all set operations that do not modify it (union, intersection, difference) but has no add/remove. Because it is hashable, a frozenset can be used as a dictionary key or as an element of another set.

#### EXAMPLE CODE

```
fs = frozenset([1, 2, 3, 3])
print(fs, type(fs))
print(fs | frozenset([4]))      # operations return new frozensets
print(fs & {2, 3, 9})

try:
    fs.add(5)
except AttributeError as e:
    print("Error:", e)

d = {frozenset([1, 2]): "pair"}   # usable as dict key
print(d[frozenset([2, 1])])
nested = {frozenset([1]), frozenset([2])}
print(len(nested))
```

#### OUTPUT

```
frozenset({1, 2, 3}) <class 'frozenset'>
frozenset({1, 2, 3, 4})
frozenset({2, 3})
Error: 'frozenset' object has no attribute 'add'
pair
2
```

## Python Dictionary

A dictionary stores **key: value** pairs. Keys must be unique and immutable; values can be anything. Since Python 3.7 dictionaries preserve insertion order. Methods: *get*, *keys*, *values*, *items*, *update*, *pop*, *popitem*, *setdefault*, *clear*, *copy*. Dictionary comprehensions are also supported.

### EXAMPLE CODE

```
student = {"name": "Asha", "age": 20, "course": "CS"}
print(student["name"], student.get("grade", "N/A"))
student["age"] = 21          # update
student["city"] = "Hyderabad" # add
del student["course"]        # delete
print(student)

for k, v in student.items():
    print(k, "->", v)
print(list(student.keys()), list(student.values()))
print("name" in student)

student.update({"age": 22, "marks": 88})
print(student.pop("city"), student)

squares = {n: n * n for n in range(1, 5)}
print(squares)

# word frequency
freq = {}
for w in "to be or not to be".split():
    freq[w] = freq.get(w, 0) + 1
print(freq)
```

### OUTPUT

```
Asha N/A
{'name': 'Asha', 'age': 21, 'city': 'Hyderabad'}
name -> Asha
age -> 21
city -> Hyderabad
['name', 'age', 'city'] ['Asha', 21, 'Hyderabad']
True
Hyderabad {'name': 'Asha', 'age': 22, 'marks': 88}
{1: 1, 2: 4, 3: 9, 4: 16}
{'to': 2, 'be': 2, 'or': 1, 'not': 1}
```

## Python range

*range()* generates an **immutable sequence of integers** lazily (it does not store all numbers). Forms: *range(stop)*, *range(start, stop)*, *range(start, stop, step)*. The stop value is **exclusive**. A negative step counts downwards.

### EXAMPLE CODE

```
print(list(range(5)))          # 0..4
print(list(range(2, 8)))       # 2..7
print(list(range(0, 20, 5)))   # step 5
print(list(range(10, 0, -3)))  # counting down
r = range(1, 6)
print(len(r), r[0], r[-1], 3 in r)
print(sum(range(1, 101)))      # 1 to 100

for i in range(1, 4):
    print("Row", i)
```

### OUTPUT

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6, 7]
[0, 5, 10, 15]
[10, 7, 4, 1]
5 1 5 True
```

5050

Row 1

Row 2

Row 3

# Unit 4: Flow Control and Conditional Statements

## Python if...else...elif

Conditional statements run code only when a condition is true. *if* checks the first condition, *elif* checks further conditions, and *else* runs when none are true. Conditions can be combined with *and/or/not*, and nested. A one-line **ternary expression** is *value\_if\_true if condition else value\_if\_false*.

### EXAMPLE CODE

```
marks = 72
if marks >= 90:
    grade = "A"
elif marks >= 75:
    grade = "B"
elif marks >= 60:
    grade = "C"
else:
    grade = "Fail"
print("Grade:", grade)

num = -4
if num > 0:
    print("positive")
elif num == 0:
    print("zero")
else:
    print("negative")

age = 20
status = "adult" if age >= 18 else "minor"      # ternary
print(status)

year = 2024
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print(year, "is a leap year")
```

### OUTPUT

```
Grade: C
negative
adult
2024 is a leap year
```

## Python for Loop

A *for* loop iterates over any sequence (list, string, tuple, dictionary, range). Use *range()* for a fixed number of repetitions, *enumerate()* to get index and value, and *zip()* to loop over two sequences together. Loops can be nested. A *for...else* block runs only if the loop finished without *break*.

#### EXAMPLE CODE

```
for fruit in ["apple", "banana", "cherry"]:
    print(fruit)

for ch in "Hi":
    print(ch, end=" ")
print()

for i, v in enumerate(["a", "b", "c"], start=1):
    print(i, v)

for n, s in zip([1, 2, 3], ["one", "two", "three"]):
    print(n, s)

# nested loop - right triangle pattern
for i in range(1, 5):
    print(" " * i)

# for...else
for n in range(2, 5):
    if n == 10:
        break
else:
    print("loop completed without break")
```

#### OUTPUT

```
apple
banana
cherry
H i
1 a
2 b
3 c
1 one
2 two
3 three
*
**
***
****
loop completed without break
```

## Python while Loop

A *while* loop repeats as long as its condition is true. Make sure something inside the loop changes the condition, otherwise it runs forever. It is best when the number of iterations is not known in advance. Like *for*, it may have an *else* block.

#### EXAMPLE CODE

```
i = 1
while i <= 5:
    print(i, end=" ")
    i += 1
print()

# sum of digits
n, total = 4321, 0
while n > 0:
    total += n % 10
    n //= 10
print("Digit sum:", total)

# reverse a number
n, rev = 1234, 0
while n:
    rev = rev * 10 + n % 10
    n //= 10
print("Reversed:", rev)
```

```

count = 3
while count > 0:
    print("Countdown", count)
    count -= 1
else:
    print("Liftoff!")

```

#### OUTPUT

```

1 2 3 4 5
Digit sum: 10
Reversed: 4321
Countdown 3
Countdown 2
Countdown 1
Liftoff!

```

## Python break and continue

**break** terminates the nearest enclosing loop immediately. **continue** skips the rest of the current iteration and jumps to the next one. Both work in *for* and *while* loops.

#### EXAMPLE CODE

```

for n in range(1, 10):
    if n == 5:
        break                # stop the loop at 5
    print(n, end=" ")
print()

for n in range(1, 8):
    if n % 2 == 0:
        continue            # skip even numbers
    print(n, end=" ")
print()

# search example
for x in [4, 8, 15, 16]:
    if x == 15:
        print("Found 15")
        break
else:
    print("Not found")

```

#### OUTPUT

```

1 2 3 4
1 3 5 7
Found 15

```

## Python pass

**pass** is a null statement: it does nothing. It is used as a placeholder where Python requires a statement syntactically, for example in empty functions, classes, or conditions that you will write later.

#### EXAMPLE CODE

```

def todo():
    pass                    # implement later

class Empty:
    pass

for i in range(3):
    if i == 1:
        pass                # nothing to do now
    else:
        print("Processing", i)

todo()
print(type(Empty()))

```

## OUTPUT

```
Processing 0  
Processing 2  
<class '__main__.Empty'>
```

# Unit 5: Python Functions

## Python Function

A function is a reusable block of code defined with *def*. It can take parameters and return a value using *return* (without return it gives *None*). Functions can return multiple values as a tuple. Always add a docstring.

### EXAMPLE CODE

```
def add(a, b):
    """Return the sum of a and b."""
    return a + b

def stats(nums):
    return min(nums), max(nums), sum(nums) / len(nums)

def hello():
    print("Hello")

print(add(3, 4))
lo, hi, avg = stats([4, 8, 6])
print(lo, hi, avg)
print(hello())          # prints Hello, then None
print(add.__doc__)
```

### OUTPUT

```
7
4 8 6.0
Hello
None
Return the sum of a and b.
```

## Function Arguments

Python supports: **positional** arguments (matched by order), **keyword** arguments (matched by name), **default** arguments (used when value is not passed), **\*args** (any number of positional arguments, collected as a tuple) and **\*\*kwargs** (any number of keyword arguments, collected as a dictionary). Beware: never use a mutable object as a default value.

### EXAMPLE CODE

```
def intro(name, age=18, city="Delhi"):
    print(name, age, city)

intro("Asha")          # defaults used
intro("Ravi", 25)      # positional
intro(city="Pune", name="Meena") # keyword (any order)

def total(*args):
    print(args, sum(args))
total(1, 2, 3, 4)

def profile(**kwargs):
    for k, v in kwargs.items():
        print(k, "=", v)
profile(name="Asha", age=20)

def mix(a, b=2, *args, **kwargs):
    print(a, b, args, kwargs)
mix(1, 5, 7, 8, x=10)

nums = [1, 2, 3]
print(*nums)           # unpacking a list into arguments
```

### OUTPUT

```
Asha 18 Delhi
Ravi 25 Delhi
```

```

Meena 18 Pune
(1, 2, 3, 4) 10
name = Asha
age = 20
1 5 (7, 8) {'x': 10}
1 2 3

```

## Python Recursion

Recursion is when a function calls itself. Every recursive function needs a **base case** (stops the recursion) and a **recursive case** (moves towards the base case). Without a base case you get *RecursionError* (default limit is about 1000 calls).

### EXAMPLE CODE

```

def factorial(n):
    if n == 0 or n == 1:          # base case
        return 1
    return n * factorial(n - 1) # recursive case

def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

def sum_digits(n):
    return n if n < 10 else n % 10 + sum_digits(n // 10)

print(factorial(5))
print([fib(i) for i in range(8)])
print(sum_digits(1234))

import sys
print(sys.getrecursionlimit())

```

### OUTPUT

```

120
[0, 1, 1, 2, 3, 5, 8, 13]
10
1000

```

## Anonymous (Lambda) Function

A **lambda** is a small nameless function written in one line: *lambda arguments: expression*. It can have many arguments but only one expression, whose value is returned automatically. Commonly used with *map*, *filter*, *sorted*.

### EXAMPLE CODE

```

square = lambda x: x * x
add = lambda a, b: a + b
print(square(5), add(2, 3))

print((lambda x: x + 10)(5))          # immediately called

words = ["pear", "fig", "banana"]
print(sorted(words, key=lambda w: len(w)))

pairs = [(1, "b"), (2, "a")]
print(sorted(pairs, key=lambda p: p[1]))

check = lambda n: "even" if n % 2 == 0 else "odd"
print(check(7))

```

### OUTPUT

```

25 5
15
['fig', 'pear', 'banana']
[(2, 'a'), (1, 'b')]

```

odd

## Global, Local and Nonlocal Variables

A **local** variable is created inside a function and exists only there. A **global** variable is defined at module level and readable everywhere. To *modify* a global variable inside a function you must declare it with **global**. **nonlocal** is used in a nested function to modify a variable of the enclosing function.

### EXAMPLE CODE

```
x = 10                                # global

def read():
    print("read:", x)                # reading is fine

def change():
    global x
    x = 99                          # modifies the global
read(); change(); print(x)

def local_demo():
    y = 5                            # local only
    print(y)
local_demo()
# print(y) -> NameError, y does not exist outside

def outer():
    count = 0
    def inner():
        nonlocal count
        count += 1
        return count
    inner(); inner()
    return count
print(outer())
```

### OUTPUT

```
read: 10
99
5
2
```

## Python global Keyword

Without *global*, assigning to a name inside a function creates a new local variable and leaves the global untouched. With *global name*, the function works on the module-level variable. Use sparingly: too many globals make code hard to debug; prefer parameters and return values.

### EXAMPLE CODE

```
counter = 0

def wrong():
    counter = 5                    # creates a NEW local variable
wrong()
print("after wrong():", counter)

def right():
    global counter
    counter += 1
right(); right()
print("after right():", counter)

def create_global():
    global brand_new
    brand_new = "created inside function"
create_global()
print(brand_new)
```

```
def bad():  
    try:  
        print(counter)  
        counter = 1          # makes counter local for the whole function  
    except UnboundLocalError as e:  
        print("UnboundLocalError caught")  
bad()
```

#### OUTPUT

```
after wrong(): 0  
after right(): 2  
created inside function  
UnboundLocalError caught
```

# Unit 6: Python Object and Class (OOP)

## Python OOPs Concepts

Object-Oriented Programming organises code around **objects**. The four pillars are: **Encapsulation** (bundling data and methods, hiding internals), **Abstraction** (exposing only essential features), **Inheritance** (a child class reuses a parent class) and **Polymorphism** (same interface, different behaviour). Advantages: reusability, modularity, easier maintenance. Below, all four appear in one small program.

### EXAMPLE CODE

```
class Animal:                                # parent class
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."

class Dog(Animal):                            # inheritance
    def speak(self):                        # overriding -> polymorphism
        return "Woof"

class Cat(Animal):
    def speak(self):
        return "Meow"

for a in [Dog("Tommy"), Cat("Kitty"), Animal("Generic")]:
    print(a.name, "says", a.speak())
```

### OUTPUT

```
Tommy says Woof
Kitty says Meow
Generic says ...
```

## Classes and Objects

A **class** is a blueprint; an **object** (instance) is a real thing created from it. `__init__` is the constructor that initialises attributes; `self` refers to the current object. Attributes store data, methods define behaviour. `__str__` defines how an object prints.

### EXAMPLE CODE

```
class Student:
    def __init__(self, name, marks):
        self.name = name                # instance attributes
        self.marks = marks

    def grade(self):
        return "Pass" if self.marks >= 40 else "Fail"

    def __str__(self):
        return f"Student({self.name}, {self.marks})"

s1 = Student("Asha", 85)
s2 = Student("Ravi", 35)
print(s1, s1.grade())
print(s2, s2.grade())
s1.marks = 90                          # modify attribute
print(s1.marks, isinstance(s1, Student))
print(hasattr(s1, "name"), getattr(s1, "name"))
```

### OUTPUT

```
Student(Asha, 85) Pass
Student(Ravi, 35) Fail
90 True
True Asha
```

## Inheritance

Inheritance lets a **child (derived) class** acquire the attributes and methods of a **parent (base) class**. Types: **single, multilevel, multiple, hierarchical** and **hybrid**. *super()* calls the parent's methods/constructor. The Method Resolution Order (*\_\_mro\_\_*) decides lookup order in multiple inheritance.

### EXAMPLE CODE

```
class Person:
    def __init__(self, name):
        self.name = name
    def show(self):
        print("Name:", self.name)

class Employee(Person):
    def __init__(self, name, salary):
        super().__init__(name)
        self.salary = salary
    def show(self):
        super().show()
        print("Salary:", self.salary)

class Manager(Employee):
    def role(self):
        print(self.name, "is a manager")

m = Manager("Meena", 90000)
m.show(); m.role()

class Flyer:
    def fly(self): print("flying")
class Swimmer:
    def swim(self): print("swimming")
class Duck(Flyer, Swimmer):
    pass
d = Duck(); d.fly(); d.swim()
print([c.__name__ for c in Duck.__mro__])
```

### OUTPUT

```
Name: Meena
Salary: 90000
Meena is a manager
flying
swimming
['Duck', 'Flyer', 'Swimmer', 'object']
```

## Abstraction and Encapsulation

**Abstraction** hides implementation and shows only what is necessary. In Python it is done with the *abc* module: an abstract class (inheriting *ABC*) has *@abstractmethod* methods that child classes **must** implement; you cannot create an object of an abstract class. **Encapsulation** restricts direct access to data: *public* (name), *protected* (*\_name*, by convention) and *private* (*\_\_name*, name-mangled). Access private data through getters/setters or *@property*.

### EXAMPLE CODE

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return round(3.14159 * self.r ** 2, 2)
```

```

try:
    Shape()
except TypeError as e:
    print("Cannot instantiate abstract class")
print(Circle(3).area())

class BankAccount:
    # encapsulation
    def __init__(self, balance):
        self.__balance = balance
    # private
    def deposit(self, amt):
        if amt > 0:
            self.__balance += amt
    @property
    def balance(self):
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
print(acc.balance)
try:
    print(acc.__balance)
except AttributeError:
    print("Private attribute not directly accessible")

```

#### OUTPUT

```

Cannot instantiate abstract class
28.27
1500
Private attribute not directly accessible

```

## Overloading and Overriding

**Overriding:** a child class redefines a method of the parent with the same name and signature. **Overloading:** same name, different parameters. Python does **not** support true method overloading (the last definition wins), so it is simulated with default arguments or *\*args*. **Operator overloading** is done with magic methods such as `__add__`, `__sub__`, `__eq__`, `__lt__`, `__len__`, `__str__`.

#### EXAMPLE CODE

```

# Overriding
class Parent:
    def show(self): print("Parent show")
class Child(Parent):
    def show(self): print("Child show")
Child().show()

# Method overloading simulated with default args
class Calc:
    def add(self, a, b=0, c=0):
        return a + b + c
c = Calc()
print(c.add(1), c.add(1, 2), c.add(1, 2, 3))

# Operator overloading
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
    def __str__(self):
        return f"({self.x}, {self.y})"
p = Point(1, 2) + Point(3, 4)
print(p, p == Point(4, 6))

```

#### OUTPUT

```

Child show
1 3 6

```

(4, 6) True

# Unit 7: Python Methods, Variables and Constructors

## Instance Method, Class Method and Static Method

**Instance method:** takes *self*, works with object data. **Class method:** decorated with `@classmethod`, takes *cls*, works with class-level data and is often used as an alternative constructor. **Static method:** decorated with `@staticmethod`, takes neither *self* nor *cls*; it is a utility function that lives inside the class.

### EXAMPLE CODE

```
class Student:
    school = "ABC School"                # class variable

    def __init__(self, name):
        self.name = name

    def hello(self):                      # instance method
        return f"Hi {self.name} of {self.school}"

    @classmethod
    def change_school(cls, new):         # class method
        cls.school = new

    @classmethod
    def from_string(cls, s):             # alternative constructor
        return cls(s.split("-")[0])

    @staticmethod
    def is_pass(marks):                  # static method
        return marks >= 40

s = Student("Asha")
print(s.hello())
Student.change_school("XYZ School")
print(s.hello())
print(Student.from_string("Ravi-20").name)
print(Student.is_pass(55), s.is_pass(30))
```

### OUTPUT

```
Hi Asha of ABC School
Hi Asha of XYZ School
Ravi
True False
```

## Instance Variables and Class Variables

**Instance variables** belong to each object (usually created in `__init__` using *self*) and differ between objects. **Class variables** are defined in the class body and are **shared** by all objects. Changing a class variable through the class affects every object; assigning through an object only creates a new instance variable.

#### EXAMPLE CODE

```
class Car:
    wheels = 4                # class variable (shared)
    count = 0

    def __init__(self, brand):
        self.brand = brand    # instance variable
        Car.count += 1

c1 = Car("Tata")
c2 = Car("Honda")
print(c1.brand, c2.brand)    # different per object
print(c1.wheels, c2.wheels, Car.wheels)
print("Cars created:", Car.count)

Car.wheels = 6
print(c1.wheels, c2.wheels)  # change visible to all
c1.wheels = 3                # creates instance variable on c1 only
print(c1.wheels, c2.wheels, Car.wheels)
print(c1.__dict__)
```

#### OUTPUT

```
Tata Honda
4 4 4
Cars created: 2
6 6
3 6 6
{'brand': 'Tata', 'wheels': 3}
```

## Constructors

A constructor is a special method that runs automatically when an object is created. In Python it is `__init__(self, ...)`. Types: **default** (no extra parameters), **parameterized** (takes values) . Python allows only one `__init__` per class, so use default arguments for flexibility. `__del__` is the destructor, called when an object is destroyed.

#### EXAMPLE CODE

```
class Demo:
    def __init__(self, name="Unknown", age=0):    # default + parameterized
        self.name = name
        self.age = age
        print("Constructor called for", name)

    def __del__(self):
        print("Destructor called for", self.name)

a = Demo()                # uses defaults
b = Demo("Asha", 20)       # parameterized
print(a.name, b.name, b.age)
del b                      # triggers the destructor
```

#### OUTPUT

```
Constructor called for Unknown
Constructor called for Asha
Unknown Asha 20
Destructor called for Asha
Destructor called for Unknown
```

*The destructor of the remaining object runs automatically when the program ends.*

# Unit 8: Python Files and Exception Handling

## Python File Operations and File Handling

Files are opened with `open(filename, mode)`. Modes: **r** read, **w** write (overwrites), **a** append, **r+** read and write, **x** create, add **b** for binary (rb, wb). Always use **with open(...)** so the file closes automatically. Read with `read()`, `readline()`, `readlines()`; write with `write()`, `writelines()`. `seek()` and `tell()` move/report the file pointer. The `os` module handles files and folders.

### EXAMPLE CODE

```
with open("notes.txt", "w") as f:
    f.write("Line one\n")
    f.write("Line two\n")

with open("notes.txt", "a") as f:
    f.writelines(["Line three\n", "Line four\n"])

with open("notes.txt", "r") as f:
    print(f.read())

with open("notes.txt") as f:
    print(f.readline().strip())
    print(f.readlines())

with open("notes.txt") as f:          # best way to loop
    for i, line in enumerate(f, 1):
        print(i, line.strip())

with open("notes.txt") as f:
    f.seek(5)
    print(f.read(3), f.tell())

import os
print(os.path.exists("notes.txt"), os.path.getsize("notes.txt"))
os.remove("notes.txt")
```

### OUTPUT

```
Line one
Line two
Line three
Line four

Line one
['Line two\n', 'Line three\n', 'Line four\n']
1 Line one
2 Line two
3 Line three
4 Line four
one 8
True 39
```

## Exception Handling

An **exception** is a runtime error. Handle it with **try / except / else / finally**. *try* holds risky code; *except* catches specific errors (ZeroDivisionError, ValueError, FileNotFoundError, TypeError, KeyError, IndexError); *else* runs if no error occurred; *finally* always runs (good for cleanup). Use *raise* to throw an error yourself and create **custom exceptions** by inheriting from *Exception*.

### EXAMPLE CODE

```
def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Cannot divide by zero")
    except TypeError:
        print("Numbers only")
    else:
        print("Result:", result)
    finally:
        print("Done")

divide(10, 2)
divide(10, 0)
divide(10, "a")

try:
    open("missing.txt")
except FileNotFoundError as e:
    print("File error:", e)

try:
    int("xyz")
except (ValueError, TypeError) as e:
    print("Conversion error:", e)

class InsufficientFunds(Exception):
    pass

def withdraw(balance, amt):
    if amt > balance:
        raise InsufficientFunds(f"Balance {balance}, requested {amt}")
    return balance - amt

try:
    withdraw(100, 500)
except InsufficientFunds as e:
    print("Custom error:", e)
```

### OUTPUT

```
Result: 5.0
Done
Cannot divide by zero
Done
Numbers only
Done
File error: [Errno 2] No such file or directory: 'missing.txt'
Conversion error: invalid literal for int() with base 10: 'xyz'
Custom error: Balance 100, requested 500
```

## Unit 9: Python Advanced Topics

### Python Iterator

An **iterable** is any object you can loop over (list, string...). An **iterator** is an object that produces values one at a time. It implements `__iter__()` (returns itself) and `__next__()` (returns the next value, raises *StopIteration* at the end). Use `iter()` and `next()` to work with them manually.

#### EXAMPLE CODE

```
nums = [10, 20, 30]
it = iter(nums)
print(next(it), next(it), next(it))
try:
    next(it)
except StopIteration:
    print("No more items")
```

```
class Countdown:
    def __init__(self, start):
        self.current = start
    def __iter__(self):
        return self
    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        value = self.current
        self.current -= 1
        return value
```

```
for n in Countdown(4):
    print(n, end=" ")
```

#### OUTPUT

```
10 20 30
No more items
4 3 2 1
```

### Python Generator

A generator is a function that uses **yield** instead of **return**. It produces values lazily, one per request, and remembers its state between calls. This saves memory (ideal for huge or infinite sequences). A **generator expression** looks like a list comprehension but uses parentheses.

#### EXAMPLE CODE

```
def count_up(n):
    for i in range(1, n + 1):
        yield i

g = count_up(3)
print(next(g), next(g), next(g))

def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

f = fib()
print([next(f) for _ in range(10)])

gen = (x * x for x in range(5))    # generator expression
print(list(gen))

import sys
print(sys.getsizeof(list(range(10000))) > sys.getsizeof(range(10000)))
print(sum(x for x in range(1000000)))    # no big list built
```

#### OUTPUT

```
1 2 3
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
[0, 1, 4, 9, 16]
True
499999500000
```

## Python Decorators

A decorator is a function that **takes another function, adds extra behaviour, and returns it** without changing its source code. It is applied with the **@name** syntax above a function definition. Inside, a *wrapper* function calls the original. Use *functools.wraps* to preserve the original name and docstring. Common uses: logging, timing, authentication.

#### EXAMPLE CODE

```
import functools, time

def logger(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with {args}")
        result = func(*args, **kwargs)
        print("Returned", result)
        return result
    return wrapper

@logger
def add(a, b):
    return a + b

add(3, 4)

def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} finished")
        return result
    return wrapper

@timer
def slow():
    time.sleep(0.1)

slow()
```

```
def repeat(n):
    def deco(func):
        def wrapper(*a, **k):
            for _ in range(n):
                func(*a, **k)
            return wrapper
        return deco

@repeat(3)
def hi():
    print("Hi")
hi()
```

#### OUTPUT

```
Calling add with (3, 4)
Returned 7
slow finished
Hi
Hi
Hi
```

## lambda, map, filter and reduce

**map(func, iterable)** applies a function to every item. **filter(func, iterable)** keeps items for which the function returns True. **reduce(func, iterable)** (from *functools*) combines all items into a single value. All three are usually used with lambda. map and filter return lazy iterators, so wrap them in *list()*.

#### EXAMPLE CODE

```
from functools import reduce

nums = [1, 2, 3, 4, 5, 6]

squares = list(map(lambda x: x ** 2, nums))
evens = list(filter(lambda x: x % 2 == 0, nums))
total = reduce(lambda a, b: a + b, nums)
biggest = reduce(lambda a, b: a if a > b else b, nums)

print(squares)
print(evens)
print(total, biggest)

words = ["apple", "kiwi", "banana"]
print(list(map(len, words)))
print(list(map(lambda a, b: a + b, [1, 2], [10, 20])))
print(reduce(lambda a, b: a * b, range(1, 6))) # 5 factorial
```

#### OUTPUT

```
[1, 4, 9, 16, 25, 36]
[2, 4, 6]
21 6
[5, 4, 6]
[11, 22]
120
```

# Unit 10: Python Modules, Packages, Libraries, Regex and Datetime

## Python Modules

A **module** is a .py file containing functions, classes and variables that you can reuse. Import with *import module*, *from module import name*, *import module as alias* or *from module import \** (avoid). Use *dir(module)* to list its contents. `__name__ == '__main__'` tells whether a file is run directly or imported.

### EXAMPLE CODE

```
# Create our own module file
with open("mymath.py", "w") as f:
    f.write("""
PI = 3.14
def area(r):
    return PI * r * r
if __name__ == "__main__":
    print("run directly")
""")

import mymath
from mymath import area
import mymath as mm
print(mymath.PI, area(2), mm.area(1))

import math, random, os
print(math.sqrt(49))
print(random.choice([1, 2, 3]) in [1, 2, 3])
print("sqrt" in dir(math))
print(__name__)
```

### OUTPUT

```
3.14 12.56 3.14
7.0
True
True
__main__
```

## Python Packages

A **package** is a folder containing multiple modules plus an `__init__.py` file (it marks the folder as a package). Packages can be nested (sub-packages). Import using dot notation: *from package.module import name*. Third-party packages are installed with *pip install name*.

### EXAMPLE CODE

```
import os
os.makedirs("mypack", exist_ok=True)
open("mypack/__init__.py", "w").close()
with open("mypack/greet.py", "w") as f:
    f.write("def hello(n):\n    return 'Hello ' + n\n")
with open("mypack/maths.py", "w") as f:
    f.write("def double(x):\n    return x * 2\n")

# Structure created:
# mypack/
#   __init__.py
#   greet.py
#   maths.py

from mypack import greet
from mypack.maths import double
import mypack.greet as g
print(greet.hello("Asha"), double(21), g.hello("Ravi"))
```

## OUTPUT

```
Hello Asha 42 Hello Ravi
```

## Python Libraries

A library is a collection of modules/packages that provides ready-made functionality. **Standard library** (comes with Python): *math*, *random*, *os*, *sys*, *datetime*, *json*, *re*, *collections*, *itertools*. **Third-party** (install with pip): *NumPy* (arrays), *Pandas* (data analysis), *Matplotlib* (plots), *Requests* (HTTP), *Scikit-learn* (machine learning).

### EXAMPLE CODE

```
import random, statistics, collections, itertools, json

print(statistics.mean([2, 4, 6]), statistics.median([1, 3, 9]))
c = collections.Counter("mississippi")
print(c.most_common(2))
print(list(itertools.permutations([1, 2, 3], 2))[:3])
print(json.dumps({"a": 1, "b": [1, 2]}))
random.seed(1)
print(random.randint(1, 10) in range(1, 11))

import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr * 2, arr.mean())
```

## OUTPUT

```
4 3
[('i', 4), ('s', 4)]
[(1, 2), (1, 3), (2, 1)]
{"a": 1, "b": [1, 2]}
True
[2 4 6 8] 2.5
```

## Python Regex (re module)

Regular expressions match text patterns. Functions: **re.match()** (only at start), **re.search()** (first match anywhere), **re.findall()** (all matches as a list), **re.sub()** (replace), **re.split()** and **re.compile()**. Key symbols: **\d** digit, **\w** word char, **\s** space, **.** any char, **\*** 0 or more, **+** 1 or more, **?** optional, **{n}** exactly n, **^** **\$** start/end, **[ ]** set, **( )** group.

### EXAMPLE CODE

```
import re

text = "Call 98765 43210 or 91234 56789 before 5 PM"
print(re.findall(r"\d+", text))
print(re.search(r"\d{5}", text).group())
print(re.match(r"Call", text) is not None)
print(re.sub(r"\d", "#", "Room 204"))
print(re.split(r"[ ,]+", "a, b c,d"))

email = "asha.k@mail.com"
pattern = r"^[\\w.]+@[\\w.]+\\. [a-z]{2,}$"
print(bool(re.match(pattern, email)))
print(bool(re.match(pattern, "bad@email"))))

m = re.search(r"([\\w+])@([\\w+])\\.com", "write to ravi@site.com now")
print(m.group(1), m.group(2), m.span())
```

## OUTPUT

```
['98765', '43210', '91234', '56789', '5']
98765
True
Room ###
['a', 'b', 'c', 'd']
True
```

```
False
ravi site (9, 22)
```

## Python Date and Time

The **datetime** module provides *date*, *time*, *datetime* and *timedelta*. Use *datetime.now()* for the current moment, *strftime()* to format a datetime into text and *strptime()* to parse text into a datetime. Format codes: %d day, %m month, %Y year, %H hour (24h), %M minute, %S second, %A weekday name, %B month name. *timedelta* represents a duration and allows date arithmetic. The *time* module offers *time()* and *sleep()*.

### EXAMPLE CODE

```
from datetime import datetime, date, timedelta
import time

d = date(2025, 8, 15)
print(d, d.year, d.month, d.day, d.strftime("%A"))

dt = datetime(2025, 1, 10, 14, 30, 5)
print(dt.strftime("%d-%m-%Y %H:%M:%S"))
print(dt.strftime("%B %d, %Y"))

parsed = datetime.strptime("25/12/2024", "%d/%m/%Y")
print(parsed.date(), type(parsed))

print(d + timedelta(days=30))
print(date(2025, 12, 25) - date(2025, 1, 1))
print((date(2025, 12, 25) - date(2025, 1, 1)).days)
print(datetime.now() > dt)
print(type(time.time()))
```

### OUTPUT

```
2025-08-15 2025 8 15 Friday
10-01-2025 14:30:05
January 10, 2025
2024-12-25 <class 'datetime.datetime'>
2025-09-14
358 days, 0:00:00
358
True
<class 'float'>
```

# Unit 11: Pandas DataFrames - Introduction, Selecting and Viewing Data

## Introduction to Pandas and DataFrames

**Pandas** is the most popular Python library for data analysis. Its two core structures are the **Series** (one-dimensional labelled array) and the **DataFrame** (two-dimensional table of rows and columns, like an Excel sheet or SQL table). Install with *pip install pandas* and import as *pd*. A DataFrame can be created from a dictionary, list of lists, NumPy array or by reading a file.

### EXAMPLE CODE

```
import pandas as pd

s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s)
print(s["b"], s.mean())

data = {
    "name": ["Asha", "Ravi", "Meena"],
    "age": [25, 32, 28],
    "city": ["Pune", "Delhi", "Chennai"],
}
df = pd.DataFrame(data)
print(df)

df2 = pd.DataFrame([[1, "x"], [2, "y"]], columns=["id", "label"])
print(df2)
print(type(df), df.shape)
```

### OUTPUT

```
a    10
b    20
c    30
dtype: int64
20 20.0
   name  age  city
0  Asha   25  Pune
1  Ravi   32  Delhi
2  Meena  28  Chennai
   id label
0    1     x
1    2     y
<class 'pandas.DataFrame'> (3, 3)
```

## Viewing Data

After loading data, inspect it first. *head(n)* / *tail(n)* show the first/last rows; *shape* gives (rows, columns); *columns* and *index* list labels; *dtypes* shows column types; *info()* summarises types and non-null counts; *describe()* gives statistics for numeric columns; *value\_counts()* counts unique values; *unique()* and *nunique()* list distinct values.

### EXAMPLE CODE

```
import pandas as pd
df = pd.DataFrame({
    "name": ["Asha", "Ravi", "Meena", "John", "Priya", "Karan"],
    "dept": ["IT", "HR", "IT", "Sales", "HR", "IT"],
    "age": [25, 32, 28, 45, 38, 30],
    "salary": [50000, 42000, 58000, 61000, 45000, 52000],
})

print(df.head(3))
print(df.tail(2))
print(df.shape, list(df.columns))
print(df.dtypes)
df.info()
print(df.describe())
print(df["dept"].value_counts())
print(df["dept"].unique(), df["dept"].nunique())
```

### OUTPUT

```
   name dept  age  salary
0  Asha   IT   25   50000
1  Ravi   HR   32   42000
2  Meena  IT   28   58000
   name dept  age  salary
4  Priya   HR   38   45000
5  Karan   IT   30   52000
(6, 4) ['name', 'dept', 'age', 'salary']
name      str
dept      str
age      int64
salary   int64
dtype: object
<class 'pandas.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   name    6 non-null         str
1   dept    6 non-null         str
2   age     6 non-null         int64
3   salary  6 non-null         int64
dtypes: int64(2), str(2)
memory usage: 324.0 bytes
      age      salary
count  6.000000    6.000000
mean   33.000000  51333.333333
std     7.321202   7312.090444
min    25.000000  42000.000000
25%    28.500000  46250.000000
50%    31.000000  51000.000000
75%    36.500000  56500.000000
max    45.000000  61000.000000
dept
IT      3
HR      2
Sales   1
Name: count, dtype: int64
<StringArray>
['IT', 'HR', 'Sales']
Length: 3, dtype: str 3
```

## Selecting Data: columns, loc, iloc, filtering

Select a column with `df['col']` (Series) or several with `df[['a','b']]` (DataFrame). **loc** selects by **label** (end of slice is included). **iloc** selects by **integer position** (end excluded, like normal Python). **Boolean filtering** keeps rows that satisfy a condition; combine conditions with `&` (and), `|` (or) and put each condition in brackets. `isin()`, `between()` and `query()` make filters easier.

### EXAMPLE CODE

```
import pandas as pd
df = pd.DataFrame({
    "name": ["Asha", "Ravi", "Meena", "John", "Priya", "Karan"],
    "dept": ["IT", "HR", "IT", "Sales", "HR", "IT"],
    "age": [25, 32, 28, 45, 38, 30],
    "salary": [50000, 42000, 58000, 61000, 45000, 52000],
})

print(df["name"].head(2))
print(df[["name", "salary"]].head(2))

print(df.loc[0]) # row with label 0
print(df.loc[1:2, ["name", "age"]]) # label slice, end included
print(df.iloc[0:2, 0:2]) # position slice, end excluded
print(df.iloc[-1, 3]) # last row, 4th column

print(df[df["age"] > 30])
print(df[(df["dept"] == "IT") & (df["salary"] > 50000)])
print(df[df["dept"].isin(["HR", "Sales"])])
print(df.query("age < 30 and dept == 'IT'"))
```

### OUTPUT

```
0    Asha
1    Ravi
Name: name, dtype: str
   name  salary
0  Asha  50000
1  Ravi  42000
name      Asha
dept        IT
age         25
salary     50000
Name: 0, dtype: object
   name  age
1  Ravi   32
2  Meena  28
   name dept
0  Asha   IT
1  Ravi   HR
52000
   name  dept  age  salary
1  Ravi   HR   32  42000
3  John  Sales  45  61000
4  Priya   HR   38  45000
   name dept  age  salary
2  Meena   IT   28  58000
5  Karan   IT   30  52000
   name  dept  age  salary
1  Ravi   HR   32  42000
3  John  Sales  45  61000
4  Priya   HR   38  45000
   name dept  age  salary
0  Asha   IT   25  50000
2  Meena   IT   28  58000
```

# Unit 12: Operations, Grouping, Merging and Manipulating Data

## Data Operations and Cleaning

Common operations: **missing values** (`isnull().sum()`, `dropna()`, `fillna(value)`), **duplicates** (`duplicated()`, `drop_duplicates()`), **renaming** (`rename()`), **type change** (`astype()`), **statistics** (`sum`, `mean`, `median`, `min`, `max`, `std`, `count`) and applying functions with `apply()` or `map()`.

### EXAMPLE CODE

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name": ["Asha", "Ravi", "Ravi", "Meena"],
    "age": [25, np.nan, np.nan, 28],
    "score": ["80", "75", "75", "90"],
})
print(df.isnull().sum())
print(df.duplicated().tolist())
df = df.drop_duplicates()
df["age"] = df["age"].fillna(df["age"].mean())    # fill missing
df["score"] = df["score"].astype(int)           # change dtype
df = df.rename(columns={"score": "marks"})
print(df)
print(df["marks"].mean(), df["marks"].max(), df["marks"].std())
print(df.dropna().shape)
print(df["name"].apply(lambda s: s.upper()).tolist())
```

### OUTPUT

```
name      0
age       2
score     0
dtype: int64
[False, False, True, False]
   name  age  marks
0  Asha  25.0    80
1   Ravi  26.5    75
3  Meena  28.0    90
81.66666666666667  90  7.637626158259733
(3, 3)
['ASHA', 'RAVI', 'MEENA']
```

## Manipulating Data: columns, sorting, pivot

Add a column by assigning to a new name, delete with `drop()`, sort with `sort_values()` (and `sort_index()`), reset the index with `reset_index()`, create conditional columns with `np.where()` or `apply()`, and summarise with `pivot_table()`. `axis=0` means rows, `axis=1` means columns.

### EXAMPLE CODE

```
import pandas as pd
df = pd.DataFrame({
    "name": ["Asha", "Ravi", "Meena", "John", "Priya", "Karan"],
    "dept": ["IT", "HR", "IT", "Sales", "HR", "IT"],
    "age": [25, 32, 28, 45, 38, 30],
    "salary": [50000, 42000, 58000, 61000, 45000, 52000],
})

import numpy as np
df["bonus"] = df["salary"] * 0.10 # new column
df["level"] = np.where(df["salary"] >= 55000, "High", "Normal")
print(df.head(3))

print(df.sort_values("salary", ascending=False).head(3))
print(df.sort_values(["dept", "age"]).head(3))

df = df.drop(columns=["bonus"]) # delete column
df = df.drop(index=[0]) # delete row
print(df.reset_index(drop=True).head(2))

print(df.pivot_table(values="salary", index="dept",
                      aggfunc="mean"))
```

### OUTPUT

```
   name dept  age  salary  bonus  level
0  Asha   IT   25   50000  5000.0  Normal
1   Ravi   HR   32   42000  4200.0  Normal
2  Meena   IT   28   58000  5800.0   High
   name dept  age  salary  bonus  level
3   John Sales  45   61000  6100.0   High
2  Meena   IT   28   58000  5800.0   High
5   Karan   IT   30   52000  5200.0  Normal
   name dept  age  salary  bonus  level
1   Ravi   HR   32   42000  4200.0  Normal
4  Priya   HR   38   45000  4500.0  Normal
0  Asha   IT   25   50000  5000.0  Normal
   name dept  age  salary  level
0   Ravi   HR   32   42000  Normal
1  Meena   IT   28   58000   High
   salary
dept
HR      43500.0
IT      55000.0
Sales   61000.0
```

## Grouping Data (groupby)

**groupby()** follows split-apply-combine: split rows into groups by a column, apply a function (sum, mean, count, min, max) to each group, and combine the results. Use *agg()* to apply several functions or different functions per column, and *size()* for group counts.

### EXAMPLE CODE

```
import pandas as pd
df = pd.DataFrame({
    "name": ["Asha", "Ravi", "Meena", "John", "Priya", "Karan"],
    "dept": ["IT", "HR", "IT", "Sales", "HR", "IT"],
    "age": [25, 32, 28, 45, 38, 30],
    "salary": [50000, 42000, 58000, 61000, 45000, 52000],
})

print(df.groupby("dept")["salary"].mean())
print(df.groupby("dept")["salary"].sum())
print(df.groupby("dept").size())

print(df.groupby("dept").agg(
    avg_salary=("salary", "mean"),
    max_age=("age", "max"),
    people=("name", "count"),
))
print(df.groupby("dept")["salary"].agg(["min", "max"]))
```

### OUTPUT

```
dept
HR      43500.000000
IT      53333.333333
Sales   61000.000000
Name: salary, dtype: float64
dept
HR      87000
IT      160000
Sales   61000
Name: salary, dtype: int64
dept
HR      2
IT      3
Sales   1
dtype: int64
      avg_salary  max_age  people
dept
HR      43500.000000      38      2
IT      53333.333333      30      3
Sales   61000.000000      45      1
      min    max
dept
HR      42000  45000
IT      50000  58000
Sales   61000  61000
```

## Merging, Joining and Concatenating

**merge()** combines DataFrames on common column(s), like a SQL join. Types: **inner** (only matching keys), **left** (all from left), **right** (all from right), **outer** (everything). **concat()** stacks DataFrames vertically (axis=0) or side by side (axis=1). **join()** combines on the index.

### EXAMPLE CODE

```
import pandas as pd

emp = pd.DataFrame({"id": [1, 2, 3, 4],
                    "name": ["Asha", "Ravi", "Meena", "John"]})
sal = pd.DataFrame({"id": [2, 3, 4, 5],
                    "salary": [42000, 58000, 61000, 70000]})

print(pd.merge(emp, sal, on="id", how="inner"))
print(pd.merge(emp, sal, on="id", how="left"))
print(pd.merge(emp, sal, on="id", how="outer"))

a = pd.DataFrame({"x": [1, 2]})
b = pd.DataFrame({"x": [3, 4]})
print(pd.concat([a, b], ignore_index=True))    # stack rows
print(pd.concat([a, b], axis=1))              # side by side

left = emp.set_index("id")
right = sal.set_index("id")
print(left.join(right, how="inner"))
```

### OUTPUT

```
id  name  salary
0  2  Ravi   42000
1  3  Meena  58000
2  4  John   61000
id  name  salary
0  1  Asha    NaN
1  2  Ravi  42000.0
2  3  Meena 58000.0
3  4  John  61000.0
id  name  salary
0  1  Asha    NaN
1  2  Ravi  42000.0
2  3  Meena 58000.0
3  4  John  61000.0
4  5   NaN 70000.0
x
0  1
1  2
2  3
3  4
x  x
0  1  3
1  2  4
   name  salary
id
2  Ravi   42000
3  Meena  58000
4  John   61000
```

# Unit 13: Reading Files using Python and Pandas (CSV, JSON, Excel, XML)

## Reading CSV Files

CSV (comma separated values) is the most common data format. Use **pd.read\_csv('file.csv')**. Useful options: *sep* (delimiter), *header*, *names*, *index\_col*, *usecols*, *nrows*, *skiprows*, *na\_values*, *parse\_dates*, *encoding*. Save with *to\_csv('out.csv', index=False)*. The built-in *csv* module also works.

### EXAMPLE CODE

```
import pandas as pd

df = pd.DataFrame({"name": ["Asha", "Ravi"], "marks": [88, 76]})
df.to_csv("students.csv", index=False)          # write

data = pd.read_csv("students.csv")              # read
print(data)
print(pd.read_csv("students.csv", usecols=["name"]))
print(pd.read_csv("students.csv", nrows=1))
print(pd.read_csv("students.csv", index_col="name"))

import csv                                      # built-in csv module
with open("students.csv") as f:
    for row in csv.reader(f):
        print(row)
```

### OUTPUT

```
   name  marks
0  Asha     88
1  Ravi     76
name
0  Asha
1  Ravi
name marks
0  Asha     88
marks
name
Asha     88
Ravi     76
['name', 'marks']
['Asha', '88']
['Ravi', '76']
```

## Reading JSON Files

JSON stores data as key-value pairs and lists. **pd.read\_json('file.json')** loads it into a DataFrame; *df.to\_json()* saves it. Use *orient='records'* for a list of objects. For nested JSON use *pd.json\_normalize()*. The built-in *json* module gives *json.load()* (file to dict), *json.dump()*, *json.loads()* and *json.dumps()*.

### EXAMPLE CODE

```
import pandas as pd, json

records = [{"name": "Asha", "marks": 88}, {"name": "Ravi", "marks": 76}]
with open("data.json", "w") as f:
    json.dump(records, f)

df = pd.read_json("data.json")
print(df)

with open("data.json") as f:
    d = json.load(f)
print(d[0]["name"], type(d))

nested = [{"id": 1, "info": {"city": "Pune", "age": 25}}]
print(pd.json_normalize(nested))
print(df.to_json(orient="records"))
```

### OUTPUT

```
   name  marks
0  Asha     88
1  Ravi     76
Asha <class 'list'>
   id  info.city  info.age
0   1        Pune        25
[{"name": "Asha", "marks": 88}, {"name": "Ravi", "marks": 76}]
```

## Reading Excel Files

Use **pd.read\_excel('file.xlsx')** (needs the *openpyxl* package: *pip install openpyxl*). Options: *sheet\_name* (name, number, or None for all sheets), *header*, *usecols*, *skiprows*. Write using *df.to\_excel('out.xlsx', index=False)*; *pd.ExcelWriter* writes several sheets.

### EXAMPLE CODE

```
import pandas as pd

a = pd.DataFrame({"name": ["Asha", "Ravi"], "marks": [88, 76]})
b = pd.DataFrame({"item": ["pen", "book"], "price": [10, 150]})
with pd.ExcelWriter("report.xlsx") as w:
    a.to_excel(w, sheet_name="Students", index=False)
    b.to_excel(w, sheet_name="Items", index=False)

print(pd.read_excel("report.xlsx"))
print(pd.read_excel("report.xlsx", sheet_name="Items"))
sheets = pd.read_excel("report.xlsx", sheet_name=None)
print(list(sheets.keys()))
```

### OUTPUT

```
   name  marks
0  Asha     88
1  Ravi     76
   item  price
0   pen     10
1  book    150
['Students', 'Items']
```

## Reading XML Files

XML stores data in nested tags. Two ways: `pd.read_xml('file.xml')` (needs *lxml*, works well for flat, record-like XML) or the built-in `xml.etree.ElementTree` module, which parses the tree so you can use `getroot()`, `find()`, `findall()`, `.text` and `.attrib`.

### EXAMPLE CODE

```
xml_data = """<students>
  <student id="1"><name>Asha</name><marks>88</marks></student>
  <student id="2"><name>Ravi</name><marks>76</marks></student>
</students>"""
with open("students.xml", "w") as f:
    f.write(xml_data)

import xml.etree.ElementTree as ET
root = ET.parse("students.xml").getroot()
rows = []
for s in root.findall("student"):
    rows.append({"id": s.attrib["id"],
                "name": s.find("name").text,
                "marks": int(s.find("marks").text)})
    print(s.attrib["id"], s.find("name").text)

import pandas as pd
print(pd.DataFrame(rows))
print(pd.read_xml("students.xml"))
```

### OUTPUT

```
1 Asha
2 Ravi
   id  name  marks
0  1  Asha    88
1  2  Ravi    76
   id  name  marks
0  1  Asha    88
1  2  Ravi    76
```

# Quick Revision: Differences Asked in Exams

## List vs Tuple vs Set vs Dictionary

| Feature    | List    | Tuple   | Set         | Dictionary            |
|------------|---------|---------|-------------|-----------------------|
| Syntax     | [1, 2]  | (1, 2)  | {1, 2}      | {'a': 1}              |
| Ordered    | Yes     | Yes     | No          | Yes (insertion order) |
| Mutable    | Yes     | No      | Yes         | Yes                   |
| Duplicates | Allowed | Allowed | Not allowed | Keys unique           |
| Indexing   | Yes     | Yes     | No          | By key                |

## Other Common Differences

| Topic                                          | Difference                                                                                                                                            |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>is</i> vs <i>==</i>                         | <i>==</i> compares values; <i>is</i> compares object identity (same memory object).                                                                   |
| <i>break</i> vs <i>continue</i> vs <i>pass</i> | <i>break</i> exits the loop; <i>continue</i> skips to the next iteration; <i>pass</i> does nothing (placeholder).                                     |
| Function vs Method                             | A function is defined independently; a method is a function defined inside a class and called on an object.                                           |
| Instance vs Class vs Static method             | Instance takes <i>self</i> (object data); class takes <i>cls</i> ( <i>@classmethod</i> , class data); static takes neither ( <i>@staticmethod</i> ).  |
| Instance vs Class variable                     | Instance variables are per object (set via <i>self</i> ); class variables are shared by all objects.                                                  |
| Overloading vs Overriding                      | Overloading: same name, different parameters (simulated in Python). Overriding: child redefines a parent method.                                      |
| Iterator vs Generator                          | An iterator is a class with <i>__iter__</i> and <i>__next__</i> ; a generator is a function using <i>yield</i> that builds an iterator automatically. |
| <i>return</i> vs <i>yield</i>                  | <i>return</i> ends the function with one value; <i>yield</i> pauses it and can produce many values, resuming later.                                   |
| Module vs Package                              | A module is a single .py file; a package is a folder of modules with an <i>__init__.py</i> .                                                          |
| <i>loc</i> vs <i>iloc</i>                      | <i>loc</i> selects by label (end of slice included); <i>iloc</i> selects by integer position (end excluded).                                          |
| <i>merge</i> vs <i>concat</i> vs <i>join</i>   | <i>merge</i> combines on key columns (SQL-style); <i>concat</i> stacks along an axis; <i>join</i> combines on the index.                              |
| inner vs left vs right vs outer                | inner: matching keys only; left: all left rows; right: all right rows; outer: all rows from both.                                                     |
| <i>sort()</i> vs <i>sorted()</i>               | <i>list.sort()</i> sorts in place and returns <i>None</i> ; <i>sorted()</i> returns a new sorted list from any iterable.                              |
| <i>append</i> vs <i>extend</i>                 | <i>append</i> adds one item (even a list as a single element); <i>extend</i> adds every item of an iterable.                                          |
| <i>remove</i> vs <i>pop</i> vs <i>del</i>      | <i>remove</i> deletes by value; <i>pop</i> deletes by index and returns it; <i>del</i> deletes by index/slice or a name.                              |
| Shallow vs Deep copy                           | Shallow copy duplicates the outer object only; <i>copy.deepcopy()</i> duplicates nested objects too.                                                  |
| <i>match</i> vs <i>search</i> (re)             | <i>match</i> checks only at the start of the string; <i>search</i> scans the whole string for the first match.                                        |
| Mode <i>w</i> vs <i>a</i>                      | <i>w</i> overwrites (or creates) the file; <i>a</i> appends to the end without erasing.                                                               |
| Local vs Global vs Nonlocal                    | Local: inside a function; global: module level; nonlocal: variable of the enclosing function.                                                         |

# Practice Programs Checklist

Write each of these yourself without looking at the examples. They cover most assignment and lab questions.

- Swap two numbers; check prime; factorial; Fibonacci series; Armstrong number; GCD and LCM
- Palindrome (string and number); reverse a string; count vowels; remove duplicates from a list; second largest element
- Word frequency counter with a dictionary; set operations on two lists
- Pattern printing: right triangle, pyramid, diamond using nested loops
- Recursion: factorial, Fibonacci, sum of digits, power of a number
- Class programs: Student, BankAccount with custom exception, Shape hierarchy with abstract base class
- File programs: count words and lines, copy one file into another, handle FileNotFoundError
- Write a decorator, a generator for primes, and use map/filter/reduce on a list
- Regex: validate email and phone number, extract all numbers from a sentence
- Pandas: load a CSV, clean missing values, filter rows, group and aggregate, merge two tables, save the result